

MWORKS.Syslab 2023b

Julia 性能优化

陈久宁

苏州同元软控信息技术有限公司

2025年5月23日



TONGYUAN
Software and Control

@苏州同元软控信息技术有限公司版权所有，
版权所有，侵权必究

传播或以其他方式使用

目录

1. 导引

2. 类型稳定

3. 数值类型

4. 多重派发

5. 内存使用

本节课程代码主要用来讲解并比较性能，建议代码根据视频课程讲解进行运行

01

Julia 性能优化：导引

©苏州同元软控信息技术有限公司版权所有，未经许可，不得复印、传播或以其他方式使用

@苏州同元软控信息技术有限公司
版权所有，侵权必究

1.1 导引: Julia 是一门高性能语言

Julia

```

julia> function mysum(A)
    rst = zero(eltype(A))
    @simd for i in eachindex(A)
        rst += A[i]
    end
    return rst
end
mysum (generic function with 1 method)

julia> @btime mysum($(rand(1024, 1024)));
130.620 μs (0 allocations: 0 bytes)

```

Julia + LoopVectorization

```

julia> using LoopVectorization

julia> function mysum(A)
    rst = zero(eltype(A))
    @tturbo for i in eachindex(A)
        rst += A[i]
    end
    return rst
end
mysum (generic function with 1 method)

julia> @btime mysum($(rand(1024, 1024)));
30.425 μs (0 allocations: 0 bytes)

```

Python Numpy

```

In [1]: import numpy as np
In [2]: A = np.random.rand(1024, 1024)
In [3]: %timeit A.sum()
322 μs ± 6.45 μs per loop (mean ± std. dev. of 7

```

MATLAB

```

>> A = rand(1024, 1024);
>> f = @() sum(A);
>> timeit(f)

```

ans =

4.1252e-04

412 μs

MATLAB 多线程(N=8)

```

>> A = rand(1024, 1024);
>> f = @() sum(A);
>> timeit(f)

```

ans =

7.5822e-05

75 μs

Julia 的性能上限很高

示例见附件:codes\1-1

1.1 导引: Julia 是一门高性能语言



```
julia> function mysum(A)
    rst = zero(eltype(A))
    @simd for i in eachindex(A)
        rst += A[i]
    end
    return rst
end
mysum (generic function with 1 method)

julia> A = rand(1024, 1024);

julia> @btime mysum($A);
131.004 μs (0 allocations: 0 bytes)
```



```
julia> function mysum(A)
    rst = 0
    @simd for i in eachindex(A)
        rst += A[i]
    end
    return rst
end
mysum (generic function with 1 method)

julia> A = rand(1024, 1024);

julia> @btime mysum($A);
1.869 ms (0 allocations: 0 bytes)
```



```
julia> function mysum(A)
    rst = 0
    @simd for i in eachindex(A)
        rst += A[i]
    end
    return rst
end
mysum (generic function with 1 method)

julia> f() = rand{Bool} ? 0 : 0.5
f (generic function with 1 method)

julia> A = [f() for i in 1:1024, j in 1:1024];

julia> @btime mysum($A);
25.179 ms (1048573 allocations: 16.00 MiB)
```



```
julia> function mysum(A)
    rst = 0
    for i in 1:size(A, 1)
        for j in 1:size(A, 2)
            rst += A[i, j]
        end
    end
    return rst
end
mysum (generic function with 1 method)

julia> f() = rand{Bool} ? 0 : 0.5
f (generic function with 1 method)

julia> A = [f() for i in 1:1024, j in 1:1024];

julia> @btime mysum($A);
39.142 ms (1048576 allocations: 16.00 MiB)
```

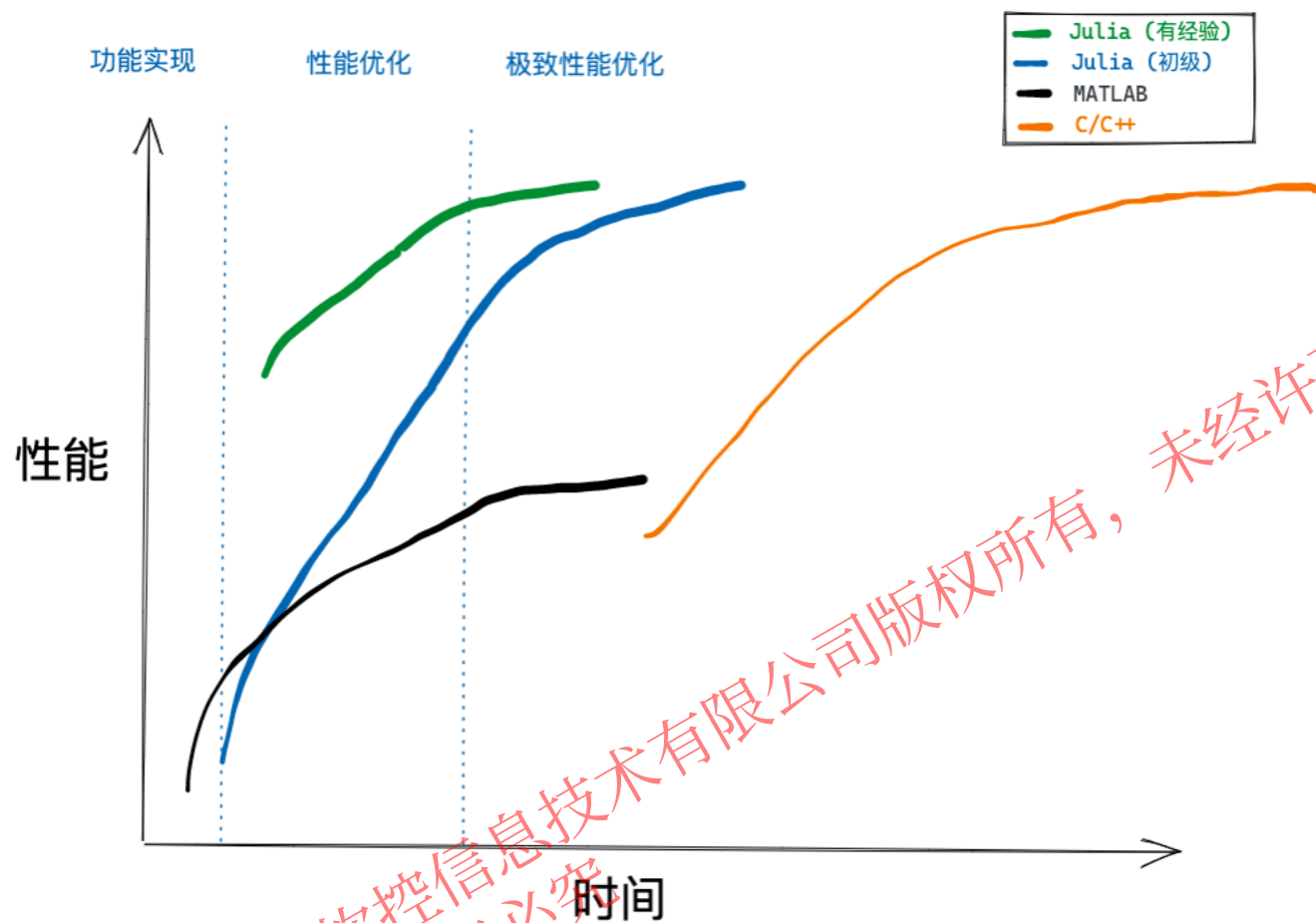
大多数 MATLAB/Python 代码的性能

大多数 Julia 初学者写的代码

但是需要经过一定的基本训练

示例见附件:codes\1-2

1.1 导引：主观经验对比



相比于 C/C++:

- Julia 可以在更短的时间内给出原型实现，快速完成建模验证工作。
- 在限定开发时间的情况下，Julia 可以引入更多的性能优化手段，从而得到更高的实际性能。

相比于 MATLAB/Python:

- Julia 可以在差不多相同的时间内给出原型实现
- Julia 实现的性能高于 MATLAB，可以处理与验证更大规模的问题

每个实际项目都存在一个交付节点：一门好的语言可以带来更高的开发效率和更高的执行效率

这是一个主观性经验性评价，并不一定适用于所有情况

1.2 导引：小结

Julia 性能优化基本原则

- 围绕 Julia 核心风格进行思考与编程
- 优先给出通用原型实现，根据需要渐进地展开性能优化
- 定位性能瓶颈，重点优化有价值的目标

Julia 核心风格

- 类型稳定
- 数值类型
- 多重派发
- 内存使用

这节课不会覆盖：

- 领域特定的算法复杂度优化
- 不太常见的高阶 Julia 优化技术
- 并行计算手段
- 编译延迟、加载延迟问题

大多数场景下，避免不必要的性能浪费就能够带来足够好的性能

02

Julia 性能优化：类型稳定

未经许可，不得复印、传播或以其他方式使用

@苏州同元软控信息技术有限公司版权所有，侵权必究

2.1 类型稳定：从 max 函数开始

max 函数在不同编程语言下的典型实现

python

```
def mymax(a, b):  
    if a > b:  
        return a  
    else:  
        return b
```

Julia

```
function mymax(a, b)  
    if a > b  
        return a  
    else  
        return b  
    end  
end
```

C/C++

```
double maxd(double a, double b){  
    if (a > b) {  
        return a;  
    } else {  
        return b;  
    }  
}
```

MATLAB

```
function c = mymax(a, b)  
    if a > b  
        c = a  
    else  
        c = b  
    end  
end
```

语言	类型	泛型支持	性能
C/C++	静态编译型语言	差	优
Python/MATLAB	动态解释型语言	优	差
Julia	动态编译型语言	优	优

问：Julia 如何在泛型的基础上做到高性能？

2.1 类型稳定：从 max 函数开始

代码编译期间只能够获得类型信息，而无法获得值信息

函数输入 mymax(1, 2) 在编译阶段是 mymax(::Int, ::Int)

Julia

```
function mymax(a, b)
    if a > b
        return a
    else
        return b
    end
end
```

mymax(::Int, ::Int)

mymax(::Float64, ::Float64)

```
julia> @code_warntype mymax(1, 2)
MethodInstance for mymax(::Int64, ::Int64)
  from mymax(a, b) @ Main REPL[1]:1
Arguments
  #self#::Core.Const(mymax)
  a::Int64
  b::Int64
Body::Int64
1 - %1 = (a > b)::Bool
   goto #3 if not %1
2 - return a
3 - return b
```

```
julia> @code_warntype mymax(1.5, 2.5)
MethodInstance for mymax(::Float64, ::Float64)
  from mymax(a, b) @ Main REPL[1]:1
Arguments
  #self#::Core.Const(mymax)
  a::Float64
  b::Float64
Body::Float64
1 - %1 = (a > b)::Bool
   goto #3 if not %1
2 - return a
3 - return b
```

实际上，编译器还可以获得一些常量信息。

2.1 类型稳定：从 max 函数开始

问：编译器如何判断 `mymax(1.5, 2)` 与 `mymax(1, 2.5)` 的返回值？

Julia

```
function mymax(a, b)
    if a > b
        return a
    else
        return b
    end
end
```

代码编译

代码执行

```
julia> typeof(mymax(1.5, 2))
Int64
julia> typeof(mymax(1, 2.5))
Float64
```

尽管实际代码运行的数值类型是确定的，但是在编译阶段，Julia 编译器进行编译优化基于的是 `Union{Float64, Int64}` 的摇摆类型来做，因此难以做到完全的性能优化。

编译器不知道输入的具体值 1.5, 2.5, 1, 2

```
julia> @code_warntype mymax(1.5, 2)
MethodInstance for mymax(::Float64, ::Int64)
  from mymax(a, b) @ Main REPL[1]:1
Arguments
  #self#::Core.Const(mymax)
  a::Float64
  b::Int64
Body::Union{Float64, Int64}
1 -- %1 = (a > b)::Bool
   └─ goto #3 if not %1
2 -- return a
3 -- return b
```

```
julia> @code_warntype mymax(1, 2.5)
MethodInstance for mymax(::Int64, ::Float64)
  from mymax(a, b) @ Main REPL[1]:1
Arguments
  #self#::Core.Const(mymax)
  a::Int64
  b::Float64
Body::Union{Float64, Int64}
1 -- %1 = (a > b)::Bool
   └─ goto #3 if not %1
2 -- return a
3 -- return b
```

@code_warntype 会用黄色/红色标记类型不稳定的情况

2.1 类型稳定：从 max 函数开始

对于类型稳定的代码来说，Julia 能够根据输入的类型来推断输出的类型

类型不稳定的代码

```
julia> function mymax(a, b)
    if a > b
        return a
    else
        return b
    end
end
mymax (generic function with 1 method)
```

类型稳定的代码

```
julia> function mymax_stable(a, b)
    a_, b_ = promote(a, b)
    if a_ > b_
        return a_
    else
        return b_
    end
end
mymax_stable (generic function with 1 method)
```

```
julia> @code_warntype mymax_stable(1.5, 2)
MethodInstance for mymax_stable(::Float64, ::Int64)
  from mymax_stable(a, b) @ Main REPL[62]:1
Arguments
  #self#::Core.Const(mymax_stable)
  a::Float64
  b::Int64
Locals
  @_4::Int64
  b_::Float64
  a_::Float64
Body::Float64
1 - %1 = Main.promote(a, b)::Tuple{Float64, Float64}
  | %2 = Base.indexed_iterate(%1, 1)::Core.PartialStruct{
Any{Float64, Core.Const(2)}}
  |   (a_ = Core.getfield(%2, 1))
  |   (@_4 = Core.getfield(%2, 2))
  |   %5 = Base.indexed_iterate(%1, 2, @_4::Core.Const(2))
  |   le{Float64, Int64}, Any{Float64, Core.Const(3)}}
  |   (b_ = Core.getfield(%5, 1))
  |   %7 = (a_ > b_)::Bool
  |   goto #3 if not %7
2 -   return a_
3 -   return b_

```

示例见附件:codes\2-1

2.1 类型稳定：从 max 函数开始

对于类型稳定的代码来说，Julia 能够根据输入的类型来推断输出的类型

类型不稳定的代码

Julia 性能优化核心建议：写类型稳定的代码

```
julia> function mymax(a, b)
    if a > b
        return a
    else
        return b
    end
end
mymax (generic function with 1 method)
```

类型稳定的代码

```
julia> function mymax_stable(a, b)
    a_, b_ = promote(a, b)
    if a_ > b_
        return a_
    else
        return b_
    end
end
mymax_stable (generic function with 1 method)
```

类型不稳定会影响后续代码的性能

```
julia> A = rand(Float64, 64, 64);
julia> B = rand(0:1, 64, 64);
julia> @btime mymax(A[1], B[1]);
53.695 ns (2 allocations: 32 bytes)
julia> @btime mymax_stable(A[1], B[1]);
55.763 ns (2 allocations: 32 bytes)
julia> @btime sum(mymax.(A, B));
136.630 μs (6209 allocations: 161.06 KiB)
julia> @btime sum(mymax_stable.(A, B));
2.826 μs (4 allocations: 32.09 KiB)
```

通过 using BenchmarkTools 来获得 @btime 性能测试工具

2.2 类型稳定: 定义

类型稳定(type stability) 理解1: 函数的返回值类型在不同情况下是不变的

```
julia> f() = rand{Bool} ? 0 : 1.0
f (generic function with 1 method)
```

```
julia> @code_warntype f()
MethodInstance for f()
  from f() in Main at REPL[6]:1
Arguments
  #self#::Core.Const(f)
Body::Union{Float64, Int64}
1 — %1 = Main.rand(Main.Bool)::Bool
   └─ goto #3 if not %1
2 — return 0
3 — return 1.0
```

```
julia> @btime f();
3.910 ns (0 allocations: 0 bytes)
```

```
julia> @btime sum($([f() for i in 1:1024]));
21.852 μs (1023 allocations: 15.98 KiB)
```

```
julia> f() = rand{Bool} ? 0.0 : 1.0
f (generic function with 1 method)
```

```
julia> @code_warntype f()
MethodInstance for f()
  from f() in Main at REPL[10]:1
Arguments
  #self#::Core.Const(f)
Body::Float64
1 — %1 = Main.rand(Main.Bool)::Bool
   └─ goto #3 if not %1
2 — return 0.0
3 — return 1.0
```

```
julia> @btime f();
6.828 ns (0 allocations: 0 bytes)
```

```
julia> @btime sum($([f() for i in 1:1024]));
69.255 ns (0 allocations: 0 bytes)
```

示例见附件:codes\2-2

2.2 类型稳定: 定义

类型稳定(type stability) 理解 2: 函数的返回值 (及中间结果) 类型可以通过函数输入的类型来唯一推断



```
julia> function f(x)
    if x < 0.5
        return Int(0)
    else
        return Float64(1.0)
    end
end
f (generic function with 2 methods)

julia> @code_warntype f(rand())
MethodInstance for f(::Float64)
  from f(x) in Main at REPL[16]:1
Arguments
  #self#::Core.Const(f)
  x::Float64
Body::Union{Float64, Int64}
1 - %1 = (x < 0.5)::Bool
└─ goto #3 if not %1
2 - %3 = Main.Int(0)::Core.Const(0)
└─ return %3
3 - %5 = Main.Float64(1.0)::Core.Const(1.0)
└─ return %5

julia> @btime sum($([f(rand()) for i in 1:1024]));
22.534 μs (1023 allocations: 15.98 KiB)
```



```
julia> function f(x)
    if x < 0.5
        return zero(x)
    else
        return one(x)
    end
end
返回与 x 类型相同的 0 或 1
f (generic function with 2 methods)

julia> @code_warntype f(rand())
MethodInstance for f(::Float64)
  from f(x) in Main at REPL[35]:1
Arguments
  #self#::Core.Const(f)
  x::Float64
Body::Float64
1 - %1 = (x < 0.5)::Bool
└─ goto #3 if not %1
2 - %3 = Main.zero(x)::Core.Const(0.0)
└─ return %3
3 - %5 = Main.one(x)::Core.Const(1.0)
└─ return %5

julia> @btime sum($([f(rand()) for i in 1:1024]));
70.824 ns (0 allocations: 0 bytes)
```



```
julia> @code_warntype f(1)
MethodInstance for f(::Int64)
  from f(x) in Main at REPL[35]:1
Arguments
  #self#::Core.Const(f)
  x::Int64
Body::Int64
1 - %1 = (x < 0.5)::Bool
└─ goto #3 if not %1
2 - %3 = Main.zero(x)::Core.Const(0)
└─ return %3
3 - %5 = Main.one(x)::Core.Const(1)
└─ return %5

julia> @code_warntype f(1.0)
MethodInstance for f(::Float64)
  from f(x) in Main at REPL[35]:1
Arguments
  #self#::Core.Const(f)
  x::Float64
Body::Float64
1 - %1 = (x < 0.5)::Bool
└─ goto #3 if not %1
2 - %3 = Main.zero(x)::Core.Const(0.0)
└─ return %3
3 - %5 = Main.one(x)::Core.Const(1.0)
└─ return %5
```

示例见附件:codes\2-3

2.3 类型稳定: 案例

输入矩阵 A 是否为空矩阵会影响返回值的类型

类型不稳定的代码

```
julia> function mysum(A)
    rst = 0
    @simd for i in eachindex(A)
        rst += A[i]
    end
    return rst
end
mysum (generic function with 1 method)

julia> A = rand(1024, 1024);

julia> @btime mysum($A);
1.869 ms (0 allocations: 0 bytes)
```

```
julia> @code_warntype mysum(rand(4, 4))
MethodInstance for mysum(::Matrix{Float64})
  from mysum(A; dims) in Main at REPL[27]:1
Arguments
  #self#::Core.Const{mysum}
  A::Matrix{Float64}
Locals
  @_3::Union{Nothing, Tuple{Int64, Int64}}
  rst::Union{Float64, Int64}
  i::Int64
Body::Union{Float64, Int64}
1 --      (rst = 0)
   %2  = Main.eachindex(A)::Base.OneTo{Int64}
   (@_3 = Base.iterate(%2))
   %4  = (@_3 === nothing)::Bool
   %5  = Base.not_int(%4)::Bool
        goto #4 if not %5
2 -- %7  = @_3::Tuple{Int64, Int64}
   (i = Core.getfield(%7, 1))
   %9  = Core.getfield(%7, 2)::Int64
   %10 = rst::Union{Float64, Int64}
   %11 = Base.getindex(A, i)::Float64
        (rst = %10 + %11)
        (@_3 = Base.iterate(%2, %9))
   %14 = (@_3 === nothing)::Bool
   %15 = Base.not_int(%14)::Bool
        goto #4 if not %15
3 --      goto #2
4 --      return rst
```

类型稳定的代码

```
julia> function mysum(A)
    rst = zero(eltype(A))
    @simd for i in eachindex(A)
        rst += A[i]
    end
    return rst
end
mysum (generic function with 1 method)

julia> A = rand(1024, 1024);

julia> @btime mysum($A);
131.004 μs (0 allocations: 0 bytes)
```

2.3 类型稳定: 案例

类型不稳定很容易从底层传播出来影响整体函数性能

```
julia> function mysum(A)
    rst = zero(eltype(A))
    @simd for i in eachindex(A)
        rst += A[i]
    end
    return rst
end
```

mysum (generic function with 1 method)

```
julia> gen_data(f, n::Int) = [f() for i in 1:n, j in 1:n]
```

gen_data (generic function with 1 method)

```
julia> f_slow() = rand{Bool} ? 0 : 1.0 ::Union{Int,Float64}
```

f_slow (generic function with 1 method)

```
julia> h(n) = mysum(gen_data(f_slow, 1024))
```

h (generic function with 1 method)

```
julia> @btime h(1024);
35.263 ms (1571575 allocations: 39.98 MiB)
```

f_slow 类型不稳定导致 h 类型不稳定且性能下降

```
julia> f_fast() = rand{Bool} ? 0.0 : 1.0
f_fast (generic function with 1 method)
```

```
julia> h(n) = mysum(gen_data(f_fast, 1024))
h (generic function with 1 method)
```

```
julia> @btime h(1024);
5.317 ms (2 allocations: 8.00 MiB)
```

示例见附件:codes\2-4

2.3 类型稳定: 案例

如果实在要引入类型不稳定, 最好在 API 接口层面而非底层实现引入

输入 dims 的值决定了返回值的类型

```
julia> function mysum(A; dims=1)
    if dims==1
        rst = Array{Float64,1}(undef, size(A,2))
        for i in 1:size(A,1)
            tmp = 0.0
            for j in 1:size(A,2)
                tmp += A[i, j]
            end
            rst[i] = tmp
        end
        return rst
    else
        return sum(A)
    end
end
mysum (generic function with 1 method)
```

如果类型不稳定在外层函数封装, 那么性能损失相对较低一些

2.3 类型稳定: 案例

大部分类型不稳定来源于条件分支可能返回不同类型，但它并不一定是类型不稳定

这个条件分支仅需要类型就可以判断

```
julia> function f(x)
    if x isa AbstractFloat
        return 3.0 * x
    elseif x isa Integer
        return 2 * x
    else
        return zero(x)
    end
end
f (generic function with 1 method)
```

```
julia> g(x::AbstractFloat) = 3.0 * x
g (generic function with 1 method)
```

```
julia> g(x::Integer) = 2 * x
g (generic function with 2 methods)
```

```
julia> g(x) = zero(x)
g (generic function with 3 methods)
```

```
julia> @code_warntype f(1.0)
MethodInstance for f(::Float64)
  from f(x) @ Main REPL[50]:1
Arguments
  #self#::Core.Const(f)
  x::Float64
Body::Float64
1 -- %1 = (x isa Main.AbstractFloat)::Core.Const(true)
   └─ goto #3 if not %1
2 -- %3 = (3.0 * x)::Float64
   └─ return %3
3 -- Core.Const:(x isa Main.Integer))
   └─ Core.Const:(goto %9 if not %5))
   └─ Core.Const:(2 * x)
   └─ Core.Const:(return %7))
   └─ Core.Const:(Main.zero(x))
   └─ Core.Const:(return %9))
```

@code_llvm f(3) 与 @code_llvm g(3) 的结果是一致的

2.3 类型稳定: 案例

类型稳定并不需要你将所有地方都添加类型标记

```
julia> function mysum_v1(A::Matrix{Float64})::Float64
    rst = 0.0::Float64
    @simd for i in eachindex(A)
        rst += A[i]::Float64
    end
    return rst::Float64
end
mysum_v1 (generic function with 1 method)

julia> function mysum_v2(A)
    rst = zero(eltype(A))
    @simd for i in eachindex(A)
        rst += A[i]
    end
    return rst
end
mysum_v2 (generic function with 1 method)
```

对于类型稳定的 Julia 代码来说, 添加额外的类型标记不会让代码变得更快

```
julia> A = rand(64, 64);

julia> @btime mysum_v1($A);
237.043 ns (0 allocations: 0 bytes)

julia> @btime mysum_v2($A);
237.026 ns (0 allocations: 0 bytes)
```

经验观察: C/C++ 程序员要比 Python/MATLAB 程序员更容易写出类型稳定的代码

示例见附件:codes\2-5

2.3 类型稳定: 案例

Julia 允许一些特殊的类型不稳定行为

```
julia> function mysumf_v1(f, A)
    rst = zero(elttype(A))
    @simd for i in eachindex(A)
        rst += f(A[i])
    end
    return rst
end
mysumf_v1 (generic function with 1 method)
```

尽管 f1 类型不稳定, 但 f1 与 f2 的性能基本一致

```
julia> f1(x) = x > 0 ? x : 0
f1 (generic function with 1 method)

julia> f2(x) = x > 0 ? x : zero(x)
f2 (generic function with 1 method)

julia> A = rand(64, 64);

julia> @btime mysumf_v1(f1, $A);
248.929 ns (0 allocations: 0 bytes)

julia> @btime mysumf_v1(f2, $A);
249.301 ns (0 allocations: 0 bytes)
```

如果摇摆的类型均为具体类型, 并且类型的可能性非常少, 那么 Julia 会使用 Union-Split 方案来优化这种类型不稳定导致的性能开销。

```
julia> @code_warntype f1(A[1])
MethodInstance for f1(::Float64)
from f1(x) @ Main REPL[28]:1
Arguments
#self#::Core.Const(f1)
x::Float64
Body::Union{Float64, Int64}
1 — %1 = (x > 0)::Bool
   └─ goto #3 if not %1
2 — return x
3 — return 0
```

Union-splitting: what it is, and why you should care: <https://julialang.org/blog/2018/08/union-splitting/>

示例见附件:codes\2-6

2.3 类型稳定: 案例

Julia 允许一些特殊的类型不稳定行为

```
julia> f1(x) = x > 0 ? x : 0
f1 (generic function with 1 method)

julia> f2(x) = x > 0 ? x : zero(x)
f2 (generic function with 1 method)

julia> A = rand(64, 64);

julia> @btime mysumf_v1(f1, $A);
248.929 ns (0 allocations: 0 bytes)

julia> @btime mysumf_v1(f2, $A);
249.301 ns (0 allocations: 0 bytes)
```

```
julia> f3(x) = x > 0 ? x : []
f3 (generic function with 1 method)

julia> @code_warntype f3(A[1])
MethodInstance for f3(::Float64)
  from f3(x) @ Main REPL[44]:1
Arguments
  #self#::Core.Const(f3)
  x::Float64
Body::Union{Float64, Vector{Any}}
1 - %1 = (x > 0)::Bool
  |   goto #3 if not %1
2 -   return x
3 - %4 = Base.vect()::Vector{Any}
  |   return %4

julia> @btime mysumf_v1(f3, $A);
3.859 μs (0 allocations: 0 bytes)
```

摇摆类型中包含抽象类型

```
julia> function f4(x)
    if x > 0
        return x
    elseif x > -1
        return Int32(0)
    elseif x > -2
        return Float32(0)
    else
        return Float16(0)
    end
end
f4 (generic function with 1 method)

julia> @code_warntype f4(A[1])
MethodInstance for f4(::Float64)
  from f4(x) @ Main REPL[47]:1
Arguments
  #self#::Core.Const(f4)
  x::Float64
Body::Any
1 - %1 = (x > 0)::Bool
  |   goto #3 if not %1
2 -   return x
3 - %4 = (x > -1)::Bool
  |   goto #5 if not %4
4 - %6 = Main.Int32(0)::Core.Const(0)
  |   return %6
5 - %8 = (x > -2)::Bool
  |   goto #7 if not %8
6 - %10 = Main.Float32(0)::Core.Const(0.0f0)
  |   return %10
7 - %12 = Main.Float16(0)::Core.Const(Float16(0.0))
  |   return %12

julia> @btime mysumf_v1(f4, $A);
86.277 μs (8192 allocations: 128.00 KiB)
```

摇摆类型可能性过多，
编译器放弃优化

如果摇摆的类型均为具体类型，并且类型的可能性非常少，那么 Julia 会使用 Union-Split 方案来优化这种类型不稳定导致的性能开销。

Union-splitting: what it is, and why you should care: <https://julialang.org/blog/2018/08/union-splitting/>

2.4 类型稳定: 小结

判断代码类型稳定的思路:

- 仅通过输入的类型（而非输入的值）能否推断输出以及中间结果的类型
- 使用 `@code_warntype` 来验证类型不稳定行为

类型不稳定会导致:

- Julia 编译器难以做充足的编译优化
- 额外的运行时类型检查和开销
- 类型不稳定往往影响的是函数调用者的性能

编程建议:

- 尽量避免类型不稳定的代码
- 类型稳定不是通过过度添加类型标记来实现的，而是通过函数的合理拆分与设计
- 在进行任何其他性能优化之前优先评估代码类型不稳定对性能的影响
- Union-Split 会优化一些特定的类型不稳定代码的性能

03

Julia 性能优化：数值类型

未经许可，不得复印、传播或以其他方式使用

@苏州同元软控信息技术有限公司版权所有，侵权必究

3.1 数值类型：具体数值类型 vs 抽象数据类型

相比于 Any、Number 等抽象类型，使用 Int、Float64 等具体类型可以带来更好的性能



```
julia> function gen_data(n)
    A = []
    for i in 1:n
        push!(A, i)
    end
    return A
end
gen_data (generic function with 2 methods)

julia> @btime sum(gen_data(1024));
30.152 μs (1512 allocations: 45.38 KiB)
```



```
julia> function gen_data(n)
    A = Int[]
    for i in 1:n
        push!(A, i)
    end
    return A
end
gen_data (generic function with 2 methods)

julia> @btime sum(gen_data(1024));
5.887 μs (6 allocations: 21.86 KiB)
```

数据类型越具体，编译器就越有可能给出好的优化策略

```
julia> [1, 2, 3]
3-element Vector{Int64}:
 1
 2
 3
```

```
julia> Any[1, 2, 3]
3-element Vector{Any}:
 1
 2
 3
```

```
julia> Float64[1, 2, 3]
3-element Vector{Float64}:
 1.0
 2.0
 3.0
```

```
julia> []
Any[]
```

```
julia> [1,2,3]
3-element Vector{Int64}:
 1
 2
 3
```

```
julia> Any[1,2,3]
3-element Vector{Any}:
 1
 2
 3
```

```
julia> Float64[1,2,3]
3-element Vector{Float64}:
 1.0
 2.0
 3.0
```

```
julia> []
Any[]
```

3.1 数值类型：具体数值类型 vs 抽象数据类型

结构体的类型声明也应该尽可能是具体的

```
julia> dist(x::Point2D, y::Point2D) = sqrt((x.x - y.x)^2 + (x.y - y.y)^2)
dist (generic function with 1 method)

julia> function f(points)
    p = Point2D(0.0, 0.0)
    out = Vector{Float64}(undef, length(points))
    for i in 1:length(points)
        out[i] = dist(p, points[i])
    end
    return out
end
f (generic function with 1 method)

julia> gen_points(n) = [Point2D(rand(), rand()) for i in 1:n]
gen_points (generic function with 1 method)
```

在没有类型标记的情况下，结构体元素类型为 Any

```
julia> struct Point2D
    x :: Any
    y :: Any
end

julia> points = gen_points(1024);

julia> @btime f($points);
119.665 μs (6658 allocations: 112.14 KiB)
```

声明结构体的元素类型是具体类型

```
julia> struct Point2D
    x::Float64
    y::Float64
end

julia> points = gen_points(1024);

julia> @btime f($points);
1.624 μs (1 allocation: 8.12 KiB)
```

```
struct Point2D
    x::Float64
    y::Float64

end
```

```
points = gen_points(1024);
```

3.1 数值类型：具体数值类型 vs 抽象数据类型

参数化结构体可以构造不同元素类型的结构体的同时，保证结构体元素类型是具体类型，从而保证性能

🚀 `Point2D{Float64}` 与 `Point2D{Int}` 的结构体元素类型都是具体类型

```
julia> struct Point2D{T}
           x::T
           y::T
       end

julia> Point2D{Int}(3, 4)
Point2D{Int64}(3, 4)

julia> Point2D{Float64}(3.5, 4.5)
Point2D{Float64}(3.5, 4.5)
```

```
struct Point2D{T}
    x::T
    y::T
end

Point2D{Int}(3, 4)
Point2D{Float64}(3.5, 4.5)
```

```
julia> points = gen_points(1024);

julia> typeof(points)
Vector{Point2D{Float64}} (alias for Array{Point2D{Float64}, 1})

julia> @btime f(points);
1.582 μs (1 allocation: 8.12 KiB)
```

🐢 参数化结构体是否高效取决于参数化后的类型

```
julia> gen_points(n) = [Point2D{Any}(rand(), rand()) for i in 1:n]
gen_points (generic function with 1 method)

julia> points = gen_points(1024);

julia> @btime f(points);
132.911 μs (8706 allocations: 144.14 KiB)
```

注：用户自行比较时，`Point2D`需要区分名称，否则易出现重定义报错

3.1 数值类型：具体数值类型 vs 抽象数据类型

函数定义上的类型标记不影响代码的编译和性能，而仅仅只影响多重派发的行为

```
julia> function mysum(A)
    rst = zero(elttype(A))
    @simd for i in eachindex(A)
        rst += A[i]
    end
    return rst
end
mysum (generic function with 1 method)
```

```
julia> function mysum_mf64(A::Matrix{Float64})
    rst = zero(elttype(A))
    @simd for i in eachindex(A)
        rst += A[i]
    end
    return rst
end
mysum_mf64 (generic function with 1 method)
```

```
julia> A = rand(64, 64);
julia> @btime mysum($A);
237.410 ns (0 allocations: 0 bytes)
julia> @btime mysum_mf64($A);
237.338 ns (0 allocations: 0 bytes)
```

3.1 数值类型：具体数值类型 vs 抽象数据类型

函数定义上的类型标记不影响代码的编译和性能，而仅仅只影响多重派发的行为

函数定义的类型标记不同

```
julia> f(x, y) = x + y
f (generic function with 1 method)

julia> g(x::Int, y::Int) = x + y
g (generic function with 1 method)
```

在多重派发的支持行为上发生差异

```
julia> f(1.0, 2.0)
3.0

julia> g(1.0, 2.0)
ERROR: MethodError: no method matching g(::Float64, ::Float64)
Stacktrace:
 [1] top-level scope
      @ REPL[6]:1
```

生成的底层代码是完全相等的 (性能也相同)

```
julia> @code_llvm f(1, 2)
; @ REPL[1]:1 within `f`
define i64 @julia_f_266(i64 signext %0, i64 signext %1) #0 {
top:
;   @ int.jl:87 within `+`
    %2 = add i64 %1, %0
;   L
    ret i64 %2
}

julia> @code_llvm g(1, 2)
; @ REPL[2]:1 within `g`
define i64 @julia_g_277(i64 signext %0, i64 signext %1) #0 {
top:
;   @ int.jl:87 within `+`
    %2 = add i64 %1, %0
;   L
    ret i64 %2
}
```

3.2 数值类型：可变结构体 vs 不可变结构体

元素类型是具体类型的不可变结构体可以允许使用栈内存，相比于堆内存具有更高的内存分配与使用效率

```
julia> dist(x::Point2D, y::Point2D) = sqrt((x.x - y.x)^2 + (x.y - y.y)^2)
dist (generic function with 1 method)

julia> function g(points)
    p = Point2D(0.0, 0.0)
    out = 0.0
    @simd for i in 1:length(points)
        out += dist(p, points[i])
    end
    return out
end
g (generic function with 1 method)

julia> gen_points(n) = [Point2D(rand(), rand()) for i in 1:n]
gen_points (generic function with 1 method)
```

🚀 不可变结构体且元素类型为具体类型

```
julia> struct Point2D
    x::Float64
    y::Float64
end

julia> points = gen_points(1024);

julia> @btime gen_points(1024);
4.219 μs (1 allocation: 16.12 KiB)

julia> @btime g(points);
736.031 ns (1 allocation: 16 bytes)
```

🐢 可变结构体且元素类型为具体类型

```
julia> mutable struct Point2D
    x::Float64
    y::Float64
end

julia> points = gen_points(1024);

julia> @btime gen_points(1024);
8.522 μs (1025 allocations: 40.12 KiB)

julia> @btime g(points);
1.456 μs (1 allocation: 16 bytes)
```

3.2 数值类型：可变结构体 vs 不可变结构体

仅包含单个具体元素的不可变结构体可以用来建立无开销抽象

 基于 Float64 封装的自定义数值类型

```
julia> struct MyFloat64 <: AbstractFloat
    x::Float64
end

julia> Base.:(+)(a::MyFloat64, b::MyFloat64) = MyFloat64(a.x + b.x)

julia> A = [MyFloat64(rand()) for i in 1:1024,j in 1:1024];

julia> @btime sum($A);
143.835 μs (0 allocations: 0 bytes)
```

 性能与原生 Float64 的计算相当

```
julia> A = [rand() for i in 1:1024,j in 1:1024];

julia> @btime sum($A);
144.664 μs (0 allocations: 0 bytes)
```

 可变结构体的性能显著下

```
julia> mutable struct MyFloat64 <: AbstractFloat
    x::Float64
end

julia> Base.:(+)(a::MyFloat64, b::MyFloat64) = MyFloat64(a.x + b.x)

julia> A = [MyFloat64(rand()) for i in 1:1024,j in 1:1024];

julia> @btime sum($A);
6.937 ms (1048575 allocations: 16.00 MiB)
```

注：用户自行比较时，MyFloat64基于Float64封装的和可变结构体需要区分名称，否则易出现重定义报错

3.2 数值类型：可变结构体 vs 不可变结构体

仅包含单个具体元素的不可变结构体可以用来建立无开销抽象

```
julia> a, b = rand(), rand();
```

原生 Float64

```
julia> @code_llvm a + b
; @ float.jl:383 within `+`
define double @"julia_+_989"(double %0, double %1) #0 {
top:
  %2 = fadd double %0, %1
  ret double %2
}
```

```
julia> a, b = MyFloat64(rand()), MyFloat64(rand())
(MyFloat64{0.1163503582332217}, MyFloat64{0.17985662916356238})
```

不可变结构体

```
julia> @code_llvm a + b
; @ REPL[2]:1 within `+`
define [1 x double] @"julia_+_968"([1 x double]* nocapture nonnull readonly align 8
top:
; @ Base.jl:38 within `getproperty`
  %2 = getelementptr inbounds [1 x double], [1 x double]* %0, i64 0, i64 0
  %3 = getelementptr inbounds [1 x double], [1 x double]* %1, i64 0, i64 0
;
; @ REPL[2]:1 within `+` @ float.jl:383
  %4 = load double, double* %2, align 8
  %5 = load double, double* %3, align 8
  %6 = fadd double %4, %5
; @ REPL[2]:1 within `+`
  %.fca.0.insert = insertvalue [1 x double] zeroinitializer, double %6, 0
  ret [1 x double] %.fca.0.insert
}
```

```
julia> @code_llvm a + b
; @ REPL[2]:1 within `+`
define nonnull {}* @"julia_+_949"({}* nonnull align 8 dereferenceable(8) %0, {}* nonnull al
top:
  %2 = call {}*** inttoptr (i64 140703225901013 to {}*** (i64)*)(i64 261) #3
; @ Base.jl:38 within `getproperty`
  %3 = bitcast {}* %0 to double*
  %4 = load double, double* %3, align 8
  %5 = bitcast {}* %1 to double*
  %6 = load double, double* %5, align 8
;
; @ REPL[2]:1 within `+` @ float.jl:383
  %7 = fadd double %4, %6
; @ REPL[2]:1 within `+`
; @ REPL[1]:2 within `MyFloat64`
  %ptls_field3 = getelementptr inbounds {}**, {}*** %2, i64 2
  %8 = bitcast {}*** %ptls_field3 to i8**
  %ptls_load45 = load i8*, i8** %8, align 8
  %9 = call noalias nonnull {}* @ijl_gc_pool_alloc(i8* %ptls_load45, i32 1392, i32 16) #4
  %10 = bitcast {}* %9 to i64*
  %11 = getelementptr inbounds i64, i64* %10, i64 -1
  store atomic i64 4608170336, i64* %11 unordered, align 8
  %12 = bitcast {}* %9 to double*
  store double %7, double* %12, align 8
;
  ret {}* %9
}
```

3.2 数值类型：可变结构体 vs 不可变结构体

可变数据类型作为不可变结构体的元素

Julia 中典型的定义一个新矩阵的方式

```
julia> struct MyDiagonal{T} <: AbstractArray{T,2}
    data::Vector{T}
end

julia> Base.size(A::MyDiagonal) = (length(A.data), length(A.data))

julia> Base.@propagate_inbounds function Base.getindex(A::MyDiagonal, i::Int, j::Int)
    return i == j ? A.data[i] : zero(eltype(A))
end

julia> A = MyDiagonal(rand(4))
4×4 MyDiagonal{Float64}:
 0.749366  0.0      0.0      0.0
 0.0      0.403765  0.0      0.0
 0.0      0.0      0.951057  0.0
 0.0      0.0      0.0      0.481639

julia> @btime A * A;
993.800 ns (6 allocations: 20.73 KiB) 通过多重派发引入对角矩阵乘法的优化实现

julia> Base.:(*)(A::MyDiagonal{T}, B::MyDiagonal{T}) where T = MyDiagonal{T}(A.data .* B.data)

julia> @btime A * A;
108.732 ns (2 allocations: 112 bytes)
```

与内置的对角矩阵的性能相当

```
julia> using LinearAlgebra

julia> A = Diagonal(rand(4))
4×4 Diagonal{Float64, Vector{Float64}}:
 0.796221  .      .      .
 .      0.399725  .      .
 .      .      0.369407  .
 .      .      .      0.678635

julia> @btime A * A;
134.494 ns (2 allocations: 112 bytes)
```

矩阵接口协议文档: <https://docs.julialang.org/en/v1/manual/interfaces/#man-interface-array>

3.2 数值类型：可变结构体 vs 不可变结构体

使用 Tuple 或 NamedTuple 来代替 Vector 或 Dict

```
julia> function mysum(pairs)
    r1, r2 = 0.0, 0.0
    @simd for p in pairs
        r1 += p[1]
        r2 += p[2]
    end
    return r1, r2
end
```

mysum (generic function with 1 method)

```
julia> pairs = [(rand(), rand()) for _ in 1:1024];
```

```
julia> @btime mysum(pairs);
259.841 ns (1 allocation: 32 bytes)
```

```
julia> pairs = [[rand(), rand()] for _ in 1:1024];
```

```
julia> @btime mysum(pairs);
989.643 ns (1 allocation: 32 bytes)
```

前提：代码是类型稳定的

```
julia> function mysum(pairs)
    r1, r2 = 0.0, 0.0
    @simd for p in pairs
        r1 += p.first
        r2 += p.second
    end
    return r1, r2
end
```

mysum (generic function with 1 method) NamedTuple

```
julia> pairs = [(; first=rand(), second=rand()) for _ in 1:1024];
```

```
julia> @btime mysum(pairs);
260.288 ns (1 allocation: 32 bytes)
```

```
julia> function mysum(pairs)
    r1, r2 = 0.0, 0.0
    @simd for p in pairs
        r1 += p[:first]
        r2 += p[:second]
    end
    return r1, r2
end
```

mysum (generic function with 1 method) Dict

```
julia> pairs = [Dict{:first=>rand(), :second=>rand()} for _ in 1:1024];
```

```
julia> @btime mysum(pairs);
11.565 μs (1 allocation: 32 bytes)
```

3.3 数值类型：小结

具体数值类型对于编译器更友好

- 矩阵元素以及结构体的元素为具体数值类型时，相关的计算性能会得到大大提高
- 可以使用参数化结构体来构造泛型的结构体定义

Julia 中大部分类型标记用于两种目的

- 结构体的元素定义需要类型标记来构造具体类型来优化结构体性能
- 函数定义处使用类型标记来构造多重派发规则

不可变结构体有可能使用更高效的栈内存

- 很多编程任务都可以使用不可变结构体来完成
- 不可变结构体的修改往往表现为创建一个新的结构体（性能有时会比可变结构体修改更好）
- 恰当的不可变结构体使用可以带来零开销抽象
- Julia 中常常使用 Tuple 或者 NamedTuple 来避免不必要的矩阵与字典使用

编译器能做的假设越强（具体类型、数值不可变），编译器越有可能生成高性能的代码。

04

Julia 性能优化：多重派发

未经许可，不得复印、传播或以其他方式使用

@苏州同元软控信息技术有限公司版权所有，侵权必究

4.1 多重派发: 函数与方法

Julia 函数 (function) 由多个方法 (method) 共同定义, 多重派发调用最匹配的方法定义

```
julia> is_number(x) = false
is_number (generic function with 1 method) 1

julia> is_number(x::Number) = true
is_number (generic function with 2 methods) 2

julia> is_number(x::Int) = true
is_number (generic function with 3 methods) 3

julia> is_number(1) 3
true

julia> is_number(1.0) 2
true

julia> is_number([2]) 1
false

julia> is_number("hello world") 1
false
```

示例见附件:codes\4-1

```
julia> f(a, b) = "f(::Any, ::Any)"
f (generic function with 1 method) 1

julia> f(a::Int, b::Int) = "f(::Int, ::Int)"
f (generic function with 2 methods) 2

julia> f(1, 2) 2
"f(::Int, ::Int)"

julia> f(3.0, 4.0) 1
"f(::Any, ::Any)"

julia> f(3, 4.0) 1
"f(::Any, ::Any)"
```

见附件:codes\4-1

Julia 内置标准库中定义了两百多种加法

```
julia> methods(+)
```

```
# 207 methods for generic function "+" from Base:
[1] +(x::Dates.CompoundPeriod, y::Dates.CompoundPeriod)
    @ Dates /Applications/Julia-1.10.app/Contents/Resources/julia/shar
tdlib/v1.10/Dates/src/periods.jl:334
[2] +(x::Dates.CompoundPeriod, y::Dates.TimeType)
    @ Dates /Applications/Julia-1.10.app/Contents/Resources/julia/shar
tdlib/v1.10/Dates/src/periods.jl:362
[3] +(x::Dates.CompoundPeriod, y::Dates.Period)
    @ Dates /Applications/Julia-1.10.app/Contents/Resources/julia/shar
tdlib/v1.10/Dates/src/periods.jl:332
```

4.2 多重派发: 类型

Julia 多重派发围绕输入的类型构建了不同的代码实现, Julia 的代码语义由类型来承载

事实: Julia 中列向量不等于行向量

```
julia> X = [1, 2, 3, 4]
4-element Vector{Int64}:
 1
 2
 3
 4

julia> Y = [1 2 3 4]
1×4 Matrix{Int64}:
 1 2 3 4

julia> X == Y
false

julia> @which X == Y
==(A::AbstractArray, B::AbstractArray)
@ Base abstractarray.jl:2925
```

示例见附件:codes\4-2

```
function (==)(A::AbstractArray, B::AbstractArray)
    if axes(A) != axes(B)
        return false
    end
    anymissing = false
    for (a, b) in zip(A, B)
        eq = (a == b)
        if ismissing(eq)
            anymissing = true
        elseif !eq
            return false
        end
    end
    return anymissing ? missing : true
end
```

```
julia> 1 == [1]
false

julia> @which 1 == [1]
==(x, y)
@ Base Base.jl:159
```

$==(x, y) = x === y$

事实: Julia 中 $1 \neq [1]$

通过 @which, @less, @edit 可以查看函数背后所调用的方法

4.3 多重派发: 特化

特化 (specialization): 通过多重派发来给出特定类型的优化实现

多重派发允许我们告诉 Julia 编译器更多的信息

通过 Diagonal 类型告诉编译器: 只有对角线有值

```
julia> using LinearAlgebra

julia> Xd = Diagonal([1, 2, 3, 4])
4×4 Diagonal{Int64, Vector{Int64}}:
 1 . . .
 . 2 . .
 . . 3 .
 . . . 4

julia> X = collect(Xd)
4×4 Matrix{Int64}:
 1 0 0 0
 0 2 0 0
 0 0 3 0
 0 0 0 4

julia> Xd == X
true
```

```
julia> Xd = Diagonal(rand(512));
julia> X = collect(Xd);

julia> @btime Xd * Xd;
398.651 ns (2 allocations: 4.14 KiB)

julia> @btime X * X;
863.433 μs (2 allocations: 2.00 MiB)

julia> Xd * Xd == X * X
true
```

```
julia> function mysum(A)
    rst = zero(eltype(A))
    @simd for i in eachindex(A)
        rst += A[i]
    end
    return rst
end

mysum (generic function with 1 method)

julia> @btime mysum(Xd);
75.197 μs (1 allocation: 16 bytes)

julia> @btime mysum(X);
30.326 μs (1 allocation: 16 bytes)
```

```
julia> mysum(X::Diagonal) = mysum(X.diag)
mysum (generic function with 2 methods)

julia> @btime mysum(Xd);
43.735 ns (1 allocation: 16 bytes)
```

示例见附件:codes\4-3

4.3 多重派发: 特化

问: 如何实现一个最佳性能的 2x2 矩阵乘法?

Day 1: 50ns 够快了, 但我们希望更快

```
julia> A = rand(2, 2);  
  
julia> B = rand(2, 2);  
  
julia> @btime A * B;  
52.310 ns (1 allocation: 96 bytes)
```

类型信息越丰富, 越有可能给出特化的高性能实现

 3x faster!

```
julia> A = Matrix22(rand(2, 2));  
  
julia> B = Matrix22(rand(2, 2));  
  
julia> @btime A * B;  
18.754 ns (1 allocation: 48 bytes)
```

示例见附件:codes\4-4

Day 2: 构造一个 2x2 的矩阵类型

```
julia> struct Matrix22{T} <: AbstractArray{T,2}  
    data::Tuple{Tuple{T,T},Tuple{T,T}}  
end  
  
julia> Base.size(A::Matrix22) = (2, 2)  
julia> Base.getindex(A::Matrix22, i::Int, j::Int) = A.data[i][j]  
  
julia> function Base.:(*)(A::Matrix22, B::Matrix22)  
    c11 = A[1, 1] * B[1, 1] + A[1, 2] * B[2, 1]  
    c12 = A[1, 1] * B[1, 2] + A[1, 2] * B[2, 2]  
    c21 = A[2, 1] * B[1, 1] + A[2, 2] * B[2, 1]  
    c22 = A[2, 1] * B[1, 2] + A[2, 2] * B[2, 2]  
    return Matrix22{typeof(c11)}((c11, c12), (c21, c22))  
end  
  
julia> A = [1 2; 3 4]  
2x2 Matrix{Int64}:  
 1  2  
 3  4  
  
julia> A * A  
2x2 Matrix{Int64}:  
 7 10  
15 22  
  
julia> Matrix22(A) * Matrix22(A)  
2x2 Matrix22{Int64}:  
 7 10  
15 22
```

矩阵接口协议文档: <https://docs.julialang.org/en/v1/manual/interfaces/#man-interface-array>

4.4 多重派发: 小结

基于多重派发, Julia 允许我们构建一个渐进式的性能优化体验:

- 第一天只需要给出通用的代码实现, 确保模型正确
- 进一步挖掘模型中的固有信息, 将这些信息以 Julia 数据类型的方式编码
- 围绕新创建的数据类型, 通过多重派发引入优化实现 —— 这只需要增加一些新的方法, 而不需要对已有的系统进行大规模重构

Julia 性能优化的核心在于渐进式优化:

- 原型实现只要能跑, 不需要过度在意它是否是编译器友好 (类型稳定、具体类型…), 也不需要过度在意它是否使用了高效的算法实现
- 在有需要的时候, 针对性地引入编译器友好的代码、或者通过多重派发引入特定的优化实现

05

Julia 性能优化：内存优化

未经许可，不得复印、传播或以其他方式使用

@苏州同元软控信息技术有限公司版权所有，侵权必究

5.1 内存优化：导引

在一门足够高性能的语言中（C/C++、Julia），内存是高性能计算必须要考虑的因素



：延列的方向进行迭代

```
julia> function mysum_colwise(A)
    rst = zero(eltype(A))
    @inbounds for j in axes(A, 2)
        @simd for i in axes(A, 1)
            rst += A[i, j]
        end
    end
    return rst
end
mysum_colwise (generic function with 1 method)

julia> A = rand(1024, 1024);

julia> @btime mysum_colwise(A);
126.697 μs (1 allocation: 16 bytes)
```



：延行的方向进行迭代

```
julia> function mysum_rowwise(A)
    rst = zero(eltype(A))
    @inbounds for i in axes(A, 1)
        for j in axes(A, 2)
            rst += A[i, j]
        end
    end
    return rst
end
mysum_rowwise (generic function with 1 method)

julia> A = rand(1024, 1024);

julia> @btime mysum_rowwise(A);
1.284 ms (1 allocation: 16 bytes)
```

示例见附件:codes\5-1

Python/MATLAB 太慢了以至于无论按照什么迭代方式，性能都差不多

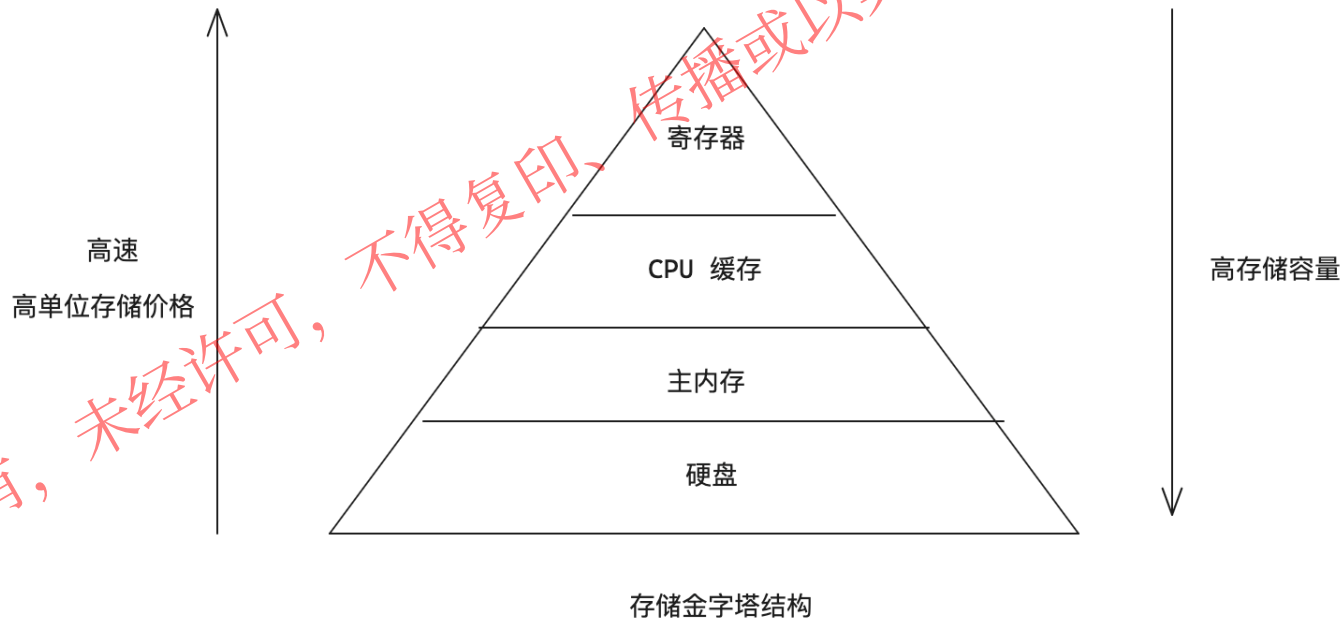
5.1 内存优化：导引

在一门足够高性能的语言中（C/C++、Julia），内存是高性能计算必须要考虑的因素

计算机存储是分层的：寄存器、CPU 三级缓存、内存、磁盘缓存、磁盘、远程存储



CPU 在处理内存中的数据（矩阵）的时候，会先将数据读取到 CPU 缓存中



CPU 在从内存中加载缓存的时候，每次会加载一段连续的内存而非单个值

CPU 缓存特性

空间局部性：内存上越相近的位置，在缓存中的概率越高
时间局部性：最近刚被使用过的值，在缓存中的概率较高

5.1 内存优化：导引

CPU 在从内存中加载缓存的时候，每次会加载一段连续的内存而非单个值



沿着数组的内存排列顺序使用矩阵，缓存命中率更高

5.1 内存优化：导引

Julia 矩阵默认延列的方向存储

```
julia> A = [1 3; 2 4]
2×2 Matrix{Int64}:
 1  3
 2  4

julia> A[:,]
4-element Vector{Int64}:
 1
 2
 3
 4

julia> A[1], A[2], A[3], A[4]
(1, 2, 3, 4)
```

A[1]	A[5]
A[2]	A[6]
A[3]	A[7]
A[4]	A[8]

```
julia> A=[1 3;2 4]
2×2 Matrix{Int64}:
 1  3
 2  4
julia> A[:,]
4-element Vector{Int64}:
 1
 2
 3
 4
julia> A[1],A[2],A[3],A[4]
(1, 2, 3, 4)
```

5.1 内存优化：导引

延内存排布的方向迭代矩阵，缓存命中率最高



：延列的方向进行迭代

```
julia> function mysum_colwise(A)
    rst = zero(eltype(A))
    @inbounds for j in axes(A, 2)
        @simd for i in axes(A, 1)
            rst += A[i, j]
        end
    end
    return rst
end
mysum_colwise (generic function with 1 method)

julia> A = rand(1024, 1024);

julia> @btime mysum_colwise(A);
126.697 μs (1 allocation: 16 bytes)
```



：延行的方向进行迭代

```
julia> function mysum_rowwise(A)
    rst = zero(eltype(A))
    @inbounds for i in axes(A, 1)
        for j in axes(A, 2)
            rst += A[i, j]
        end
    end
    return rst
end
mysum_rowwise (generic function with 1 method)

julia> A = rand(1024, 1024);

julia> @btime mysum_rowwise(A);
1.284 ms (1 allocation: 16 bytes)
```

高性能代码能够更好地利用 CPU 缓存

示例见附件:codes\5-1

5.1 内存优化：导引

延内存排布的方向迭代矩阵，缓存命中率最高

🚀：延列的方向进行迭代

```
julia> function mysum_colwise(A)
    rst = zero(eltype(A))
    @inbounds for j in axes(A, 2)
        @simd for i in axes(A, 1)
            rst += A[i, j]
        end
    end
    return rst
end
mysum_colwise (generic function with 1 method)

julia> A = rand(1024, 1024);

julia> @btime mysum_colwise(A);
126.697 μs (1 allocation: 16 bytes)
```

🚀：使用 eachindex 来延内存方向进行迭代

```
julia> function mysum(A)
    rst = zero(eltype(A))
    @inbounds @simd for i in eachindex(A)
        rst += A[i]
    end
    return rst
end
mysum (generic function with 1 method)

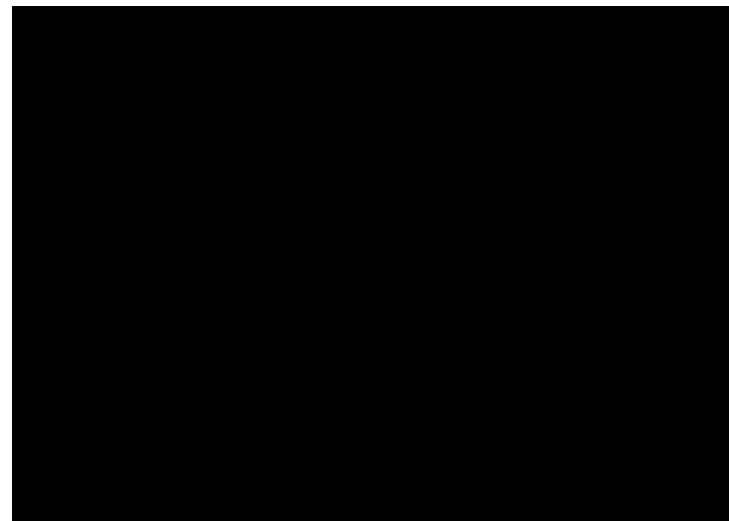
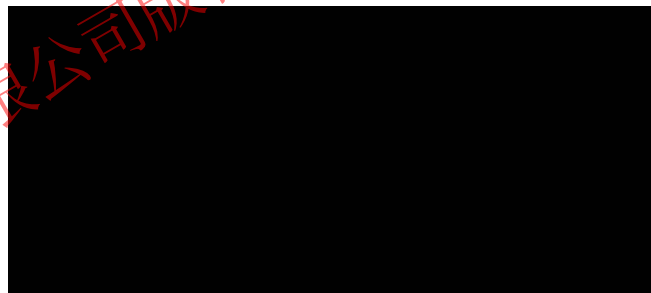
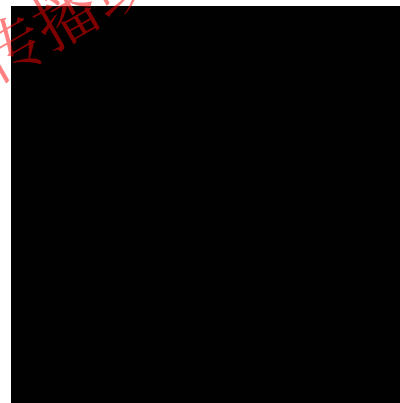
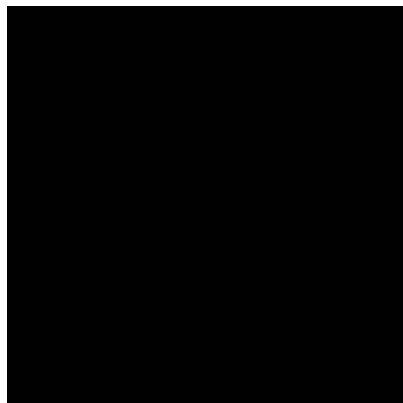
julia> A = rand(1024, 1024);

julia> @btime mysum(A);
126.405 μs (1 allocation: 16 bytes)
```

示例见附件:codes\5-1

5.1 内存优化：导引

如何（自动）找到最佳的 for 循环迭代顺序是算法实现实现极致性能的核心思路与难点



for循环迭代顺序.mp4

图片来自于 [halide](#) 文档

5.1 内存优化：导引

LoopVectorization 提供了一个简单的 for 循环优化工具

🚀：使用 eachindex 来延内存方向进行迭代

```
julia> function mysum(A)
    rst = zero(eltype(A))
    @inbounds @simd for i in eachindex(A)
        rst += A[i]
    end
    return rst
end
mysum (generic function with 1 method)

julia> A = rand(1024, 1024);

julia> @btime mysum(A);
126.405 μs (1 allocation: 16 bytes)
```

🚀🚀：自动 for 循环优化与并行

```
julia> function mysum_turbo(A)
    rst = zero(eltype(A))
    @tturbo for i in eachindex(A)
        rst += A[i]
    end
    return rst
end
mysum_turbo (generic function with 1 method)

julia> A = rand(1024, 1024);

julia> @btime mysum_turbo(A);
29.250 μs (1 allocation: 16 bytes)
```

示例见附件:codes\5-1

LoopVectorization 的使用限制较为严格，大多数时候只有底层的 for 循环才能适用

5.2 内存优化: copy vs view

Julia 矩阵取切片操作默认使用复制

Ac 发生了数据复制:
修改 Ac 不影响 A 的值

```
julia> A = [1 2; 3 4]
2×2 Matrix{Int64}:
 1  2
 3  4

julia> Ac = A[2, :]
2-element Vector{Int64}:
 3
 4

julia> Ac[1] = 0
0

julia> A
2×2 Matrix{Int64}:
 1  2
 3  4
```

Av 不发生数据复制:
修改 Av 同时影响 A 的值

```
julia> A
2×2 Matrix{Int64}:
 1  2
 3  4

julia> Av = @view A[2, :]
2-element view(::Matrix{Int64}, 2,
 3
 4

julia> Av[1] = 0
0

julia> A
2×2 Matrix{Int64}:
 1  2
 0  4
```

5.2 内存优化: copy vs view

例: 窗口大小为 3 的中值滤波算法

 : 窗口滑动时在复制与创建矩阵

```
julia> function medfilt3(v)
    out = similar(v)
    out[1] = v[1]
    out[end] = v[end]
    for i in 2:length(v)-1
        out[i] = median(v[i-1:i+1])
    end
    return out
end^C
```

```
julia> v = rand(0:255, 1024);
```

```
julia> @btime medfilt3(v);
79.033 μs (2045 allocations: 167.81 KiB)
```

 : 避免窗口滑动的复制行

```
julia> function medfilt3_view(v)
    out = similar(v)
    out[1] = v[1]
    out[end] = v[end]
    for i in 2:length(v)-1
        out[i] = median(@view v[i-1:i+1])
    end
    return out
end
```

```
medfilt3_view (generic function with 1 method)
```

```
julia> v = rand(0:255, 1024);
```

```
julia> @btime medfilt3_view(v);
49.585 μs (1023 allocations: 87.97 KiB)
```

示例见附件:codes\5-2

5.2 内存优化: copy vs view

例: 窗口大小为 3 的中值滤波算法



: 避免窗口滑动的复制行

为

```
julia> function medfilt3_view(v)
    out = similar(v)
    out[1] = v[1]
    out[end] = v[end]
    for i in 2:length(v)-1
        out[i] = median(@view v[i-1:i+1])
    end
    return out
end
```

medfilt3_view (generic function with 1 method)

```
julia> v = rand(0:255, 1024);
```

```
julia> @btime medfilt3_view(v);
49.585 μs (1023 allocations: 87.97 KiB)
```



: 避免中位数计算带来的内存分

为

```
julia> function medfilt3_v3(v)
    out = similar(v)
    out[1] = v[1]
    out[end] = v[end]
    @inbounds for i in 2:length(v)-1
        out[i] = median3(v[i-1], v[i], v[i+1])
    end
    return out
end
```

medfilt3_v3 (generic function with 1 method)

```
julia> v = rand(0:255, 1024);
```

```
julia> @btime medfilt3_v3(v);
623.262 ns (1 allocation: 8.12 KiB)
```

计算三个数之间的中位数

```
julia> function median3(a, b, c)
    return a > b ? (a < c ? a : (b > c ? b : c)) : (b < c ? b : (a > c ? a : c))
end
median3 (generic function with 1 method)
```

移除不必要的内存创建开销可以带来更高性能的代码

示例见附件:codes\5-2

5.3 内存优化：避免中间结果的矩阵创建

例：计算绝对值的和

：abs.(A) 会创建一个中间矩阵

```
julia> A = rand(1024, 1024) .- 0.5;
julia> @btime sum(abs.(A));
1.110 ms (4 allocations: 8.00 MiB)
```

：使用流式版本的 sum

```
julia> A = rand(1024, 1024) .- 0.5;
julia> @btime sum(abs, A);
151.524 μs (1 allocation: 16 bytes)
```

：实际上 abs 的中间结果只需要用标量表

```
julia> function sumabs(A)
    rst = zero(eltype(A))
    @simd for x in A
        rst += abs(x)
    end
    return rst
end
sumabs (generic function with 1 method)
julia> A = rand(1024, 1024) .- 0.5;
julia> @btime sumabs(A);
131.933 μs (1 allocation: 16 bytes)
```

示例见附件: codes\ 5-3

Julia 存在非常多高阶函数，如 map, reduce, sum, mean 用于避免手写循环

5.4 内存优化：广播

广播本质上是 for 循环，它是对典型 for 循环用法的一层抽象封装

```
julia> function add(A, B)
    @assert axes(A) == axes(B)
    C = similar(A)
    @inbounds @simd for i in eachindex(A)
        C[i] = A[i] + B[i]
    end
    return C
end

add (generic function with 1 method)

julia> A, B = rand(64, 64), rand(64, 64);

julia> add(A, B) == A .+ B
true

julia> @btime add(A, B);
1.653 μs (2 allocations: 32.05 KiB)

julia> @btime A .+ B;
2.010 μs (3 allocations: 32.08 KiB)
```

在 Julia 中，广播与 for 循环的行为与性能是一致的

MATLAB

```
C = A .* B;
D = C > 0;
```

广播

C

```
matrix C;
initial_matrix(C, m);
for (int i=0; i<m; i++){
    C[i] = A[i] * B[i];
}

matrix D;
initial_matrix(D, m);
for (int i=0; i<m; i++){
    D[i] = C[i] > 0;
}
```

对于 MATLAB/Python 来说，通过广播可以将 for 循环的展开转移到更高效的 C 层面进行，从而获得性能加速

广播本质上是一个性能优化的 tradeoff 取舍：它以更高的内存开销作为代价

算子融合问题：深度学习与大规模 AI 模型训练中的内存墙问题大多数是因为广播的额外内存开销导致的

5.4 内存优化：广播

Julia 有很好的向量化编程支持：但它的主要目的是为了简化写法而不是加速代码

Python/MATLAB 中常见的向量化风格编程

```
function f(X, Y, Z)
    M = X .> Y
    C = similar(X)
    C[M] .= X[M] .* Z[M]
    M .= .!M
    C[M] .= X[M] .+ Y[M]
    return C
end
```

但在 Julia 中更多的是标量实现+广播

```
function g(x, y, z)
    return x > y ? x * z : x + y
end
```

```
julia> using BenchmarkTools

julia> X, Y, Z = rand(4096), rand(4096), rand(4096);

julia> f(X, Y, Z) == g.(X, Y, Z)
true

julia> @btime f($X, $Y, $Z);
15.892 μs (14 allocations: 133.53 KiB)

julia> @btime g.($X, $Y, $Z);
1.563 μs (2 allocations: 32.05 KiB)
```

示例见附件:codes\5-4

5.4 内存优化：广播

Julia 的广播会尽可能多地融合连续的点运算符进一个 for 循环: 更少的内存开销和更高的性能

```
f(X, Y) = X.^2 + Y.^2
```

VS

```
g(X, Y) = X.^2 .+ Y.^2
```

背后发生的是

```
function f(X, Y)
    @assert size(X) == size(Y)

    X_rst = similar(X)
    for i in 1:length(X)
        X_rst[i] = X[i]^2
    end
    Y_rst = similar(Y)
    for i in 1:length(Y)
        Y_rst[i] = Y[i]^2
    end
    rst = similar(sin_rst)
    for i in 1:length(X)
        rst[i] = X_rst[i] + Y_rst[i]
    end
    return rst
end
```

```
julia> X, Y = rand(512, 512), rand(512, 512);
julia> f(X, Y) == g(X, Y)
true
julia> @btime f(X, Y);
636.321 μs (6 allocations: 6.00 MiB)
julia> @btime g(X, Y);
164.439 μs (2 allocations: 2.00 MiB)
```

背后发生的是

```
function f(X, Y)
    @assert size(X) == size(Y)

    rst = similar(sin_rst)
    for i in 1:length(X)
        rst[i] = X[i]^2 + Y[i]^2
    end
    return rst
end
```

更多阅读: [More Dots: Syntactic Loop Fusion in Julia](#)

5.5 内存优化：综合案例

例：计算 kurtosis 峰值

Matlab: 600 μ s

v1

Julia: 200 μ s

```
1 function kurtosis(A)
2     mean(abs.(A).^4) / (mean(abs.(A).^2))^2
3 end
```

v2 避免对 $\text{abs}(A).^2$ 的重复计算，使用 abs2

Matlab: 172 μ s

Julia: 11 μ s

```
1 function kurtosis(A)
2     tmp = abs2.(A)
3     mean(tmp.^2) / mean(tmp)^2
4 end
```

v3 使用流式版本的函数

避免先计算完所有的 $\text{abs4}(A)$ 后再求平均

Julia: 3.275 μ s

```
1 function kurtosis(A)
2     mean(abs4, A) / mean(abs2, A)^2
3 end
4 abs4(x) = abs2(x)^2
```

v4 计算本质上可以在一个循环内完成
使用 for 循环重写

Julia: 1.620 μ s

```
1 function kurtosis(A)
2     E = zero{Float64}
3     S = zero{Float64}
4     @inbounds @simd for i in eachindex(A)
5         x = A[i]
6         x2 = abs2(x)
7         x4 = abs2(x2)
8         E += x2
9         S += x4
10    end
11    return E * length(A) / S^2
12 end
```

将计算融合在一个循环中从而获得
更少的矩阵创建和更高的缓存利用率

示例见附件:codes\5-5

5.5 内存优化：小结

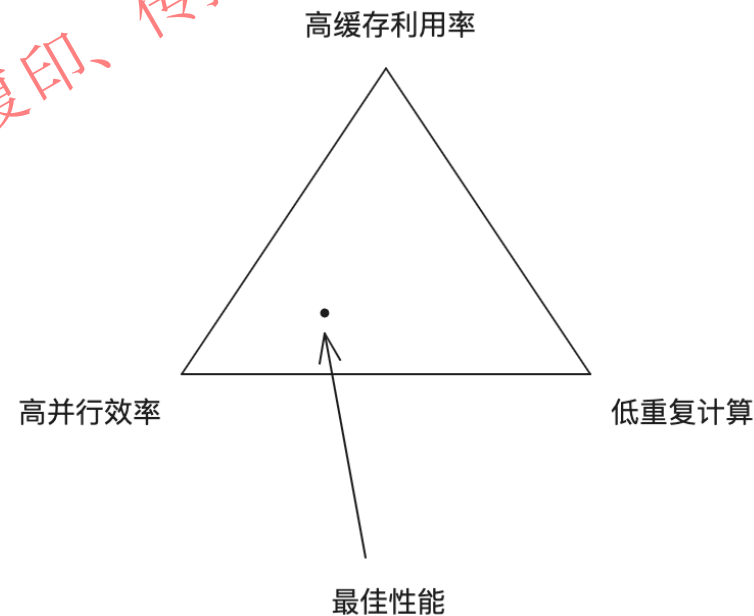
性能要点：

关注内存分配与使用可以极大地提升代码性能

- for 循环的迭代顺序会影响内存的读写模式
- 不必要的矩阵创建会极大影响性能：
 - 使用 view 而非 copy
 - 预先分配内存和复用内存
- 在 Julia 中广播并不一定意味着高效

一些没有讨论的点：

- 通过内存对齐来优化缓存利用率
- 避免重复运算
- 内存布局：SoA (Struct of Array) 与 AoS (Array of Struct)
- 大部分 GPU 运算要求我们必须使用向量化编程
- 在进行文件读写时引入 IOBuffer



帮助与反馈

用户可以根据自己需求登录12345服务-查看视频课程、登录社区、提出建议



MWORKS

MATLAB 替代工具

—— 科学计算与通用仿真平台 ——

咨询求助入口

1. 点击“**视频课程**”可查看MWORKS入门到精通课程。

2. 工作日09:00~18:30, 点击“**客服**”可接通同元官方客服, 为您答疑解惑。

3. 需求/缺陷/非工作时间, 可点击“**反馈**”按钮提单。




课程入口